

Cómo desarrollamos el MATP

Spec Driven Development con OpenSpec y GitHub Flow, con Claude Code u OpenCode como agente de desarrollo.



Sistema de Gestión y Digitalización de Colecciones Museográficas

Museo de Artes y Tradiciones Populares «Luis Repetto Málaga» · Dirección de Cultura PUCP

Grupo 4 · 1INF47 Desarrollo de Programas 2 · PUCP 2026-2

Septiembre 2026

Al terminar este entrenamiento deberías poder:

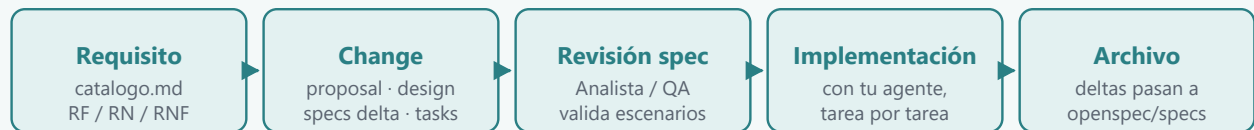
- leer un change de OpenSpec y saber exactamente qué hay que construir y cómo se verifica;
- implementarlo en su propia rama con Claude Code u OpenCode, sin salirte del alcance;
- abrir un Pull Request que pase CI y la revisión a la primera, y archivar el change después del merge.

SECCIÓN 1

La idea en una página

Trabajamos con dos ciclos que encajan: uno decide **qué** se construye (la spec) y otro controla **cómo** llega ese trabajo a main (ramas y Pull Requests). Ninguna funcionalidad entra a main sin pasar por los dos.

QUÉ construimos · Spec Driven Development (OpenSpec)



CÓMO lo integramos · GitHub Flow (Git + GitHub)



La regla de oro

1 change = 1 rama = 1 Pull Request. Antes de escribir código ejecuta `openspec list` y trabaja solo dentro del change asignado a tu célula en `docs/ownership.md`. Si lo que quieres hacer no está en ningún change, primero se propone el change. Spec primero, código después.

Qué necesitas instalado

- Git, Node.js 24, Python 3.14, Docker Desktop y OpenSpec CLI (`npm install -g @fission-ai/openspec@latest`).
- Un agente de desarrollo: **Claude Code** o, como alternativa gratuita, **OpenCode** (sección 3).
- El entorno del proyecto listo según `docs/ONBOARDING.md`: `npm install`, `npm run setup` y `npm test` en verde.

SECCIÓN 2

Spec Driven Development con OpenSpec

La spec no es documentación que se escribe al final: es el **contrato** que se escribe primero. De ella salen el código, las pruebas y la revisión. La usamos porque somos diez personas en cinco células, con reglas de negocio que no se pueden romper (código I inmutable, comodato sin código I, nunca borrar, IA con aprobación humana), y porque los agentes de IA trabajan mucho mejor con una spec precisa.

Carpeta	Qué es
openspec/specs/	La verdad actual : qué hace hoy el sistema. No se edita a mano; cambia solo al archivar un change.
openspec/changes/<change>/	Una propuesta en curso. Es donde trabaja tu célula.

Los cuatro archivos de un change (léelos en este orden)

Archivo	Responde a	Qué buscar
proposal.md	¿Por qué y qué?	Problema, alcance y lo que queda fuera.
design.md	¿Cómo?	Decisiones (D1, D2...) e interfaces que expone o consume.
specs/.../spec.md	¿Qué debe cumplirse?	Requirements con MUST/SHALL y escenarios WHEN/THEN.
tasks.md	¿En qué pasos?	Checklist - [] con cómo verificar cada tarea.

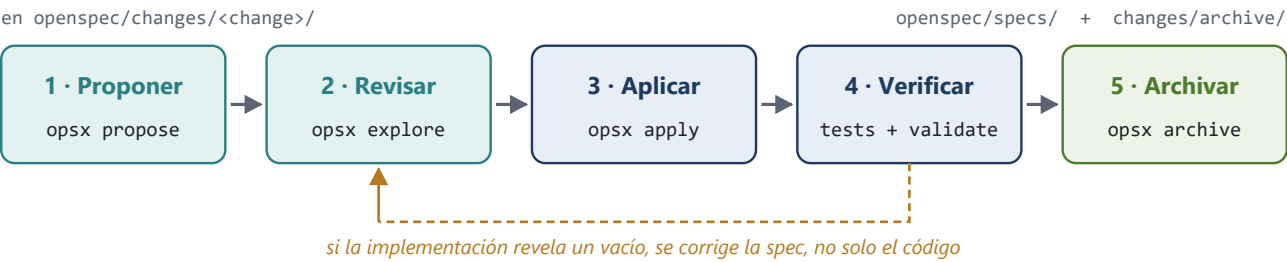
Cómo se lee un requirement

```
### Requirement: Validación previa sin persistencia
La API MUST ofrecer una operación de validación de la ficha [...] y SHALL NOT
escribir ningún dato ni auditoría al validar. (RF-043, RNF-010)

#### Scenario: La validación no persiste
- **WHEN** se valida una ficha completa y correcta
- **THEN** no se crea ninguna pieza ni registro de auditoría
```

- **MUST / SHALL**: lo obligatorio. (**RF-043...**): trazabilidad a `docs/requisitos/catalogo.md`, obligatoria.
- **Cada escenario WHEN/THEN se convierte en al menos una prueba**. Siempre hay uno de error o caso límite.
- **[SUPUESTO: ...]** marca lo que no sabemos del museo; se registra en `docs/preguntas-contraparte.md`. Nunca se inventan datos.

Ciclo de vida de un change



La spec manda sobre el código

Si al implementar descubres que la spec está mal o incompleta, no la «arregles» solo en el código: avisa al revisor de specs de tu célula y corrige el change. Código y spec deben decir lo mismo cuando abres el PR.

SECCIÓN 3

Tu agente: Claude Code u OpenCode

Los dos sirven para el mismo flujo. **Claude Code** es la opción principal; **OpenCode** es la alternativa gratuita y de código abierto (permite usar modelos gratuitos o tu propia API key). Ambos leen `CLAUDE.md` al iniciar, así que conocen las reglas del proyecto.

Para...	Claude Code	OpenCode	Qué hace
Aclarar dudas	<code>/opsx:explore</code>	<code>/opsx-explore</code>	Investiga la spec y el código. No escribe código.
Proponer un change	<code>/opsx:propose</code>	<code>/opsx-propose</code>	Crea los cuatro archivos del change.
Implementar	<code>/opsx:apply</code>	<code>/opsx-apply</code>	Recorre <code>tasks.md</code> , implementa, prueba y marca - [x].
Archivar	<code>/opsx:archive</code>	<code>/opsx-archive</code>	Pasa los deltas a <code>openspec/specs/</code> . Solo tras el merge.
Corregir el change	<code>/opsx:update</code>	pídeselo en el chat	Ajusta proposal, design, specs y tasks sin tocar código.

Observa la diferencia: en Claude Code los comandos llevan **dos puntos** (`/opsx:apply`); en OpenCode, **guion** (`/opsx-apply`). Los de Claude Code están en `.claude/` y los de OpenCode en `.opencode/`; ambos se generan con `openspec init --tools claude,opencode` (versión 1.13.0, la misma de CI).

Alternativa: sin comandos slash, con prompts

Los comandos slash son atajos. Si tu agente no los tiene, o prefieres escribir en lenguaje natural, pídele que use la **CLI de OpenSpec**. La clave es `openspec instructions`: devuelve las instrucciones exactas del paso, es decir, qué archivos leer, qué plantilla seguir y qué tareas faltan. Funciona igual con cualquier agente.

Paso	Qué pedirle al agente
Explorar	«Lee <code>openspec show <change></code> y los archivos del change. Explícame el alcance y las dudas que veas. No escribas código. »
Proponer	«Crea el change con <code>openspec new change <nombre></code> . Para cada artefacto (proposal, design, specs, tasks) sigue <code>openspec instructions <artefacto> --change <nombre></code> . Al final, <code>openspec validate <nombre> --strict.</code> »
Implementar	«Ejecuta <code>openspec instructions apply --change <change></code> y síguelo: implementa las tareas pendientes una por una, cada una con su prueba, y márcala - [x] en tasks.md.»
Ver avance	«Ejecuta <code>openspec status --change <change></code> y dime qué falta.»
Archivar	«El PR ya se integró. Ejecuta <code>openspec archive <change></code> -y y luego <code>openspec validate --all --strict.</code> »

Empieza cada sesión con este prompt (reemplaza <CHANGE>):

Lee CLAUDE.md. Ejecuta ``openspec show <CHANGE>`` y trabaja SOLO en ese change. Antes de codificar revisa proposal, design, specs y tasks; si ves vacíos o contradicciones, dímelo antes de implementar. Luego aplica el change tarea por tarea, con pruebas. Al terminar corre `npm run lint`, `npm test` y `openspec validate <CHANGE> --strict`. No archives hasta que el PR esté aprobado.

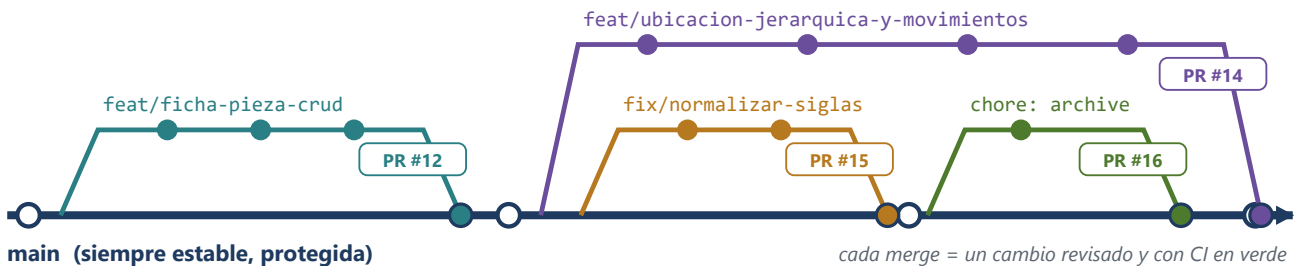
Cuatro reglas para trabajar con cualquier agente

1. **Una sesión y una rama por change.** No mezcles dos changes.
2. **Lee el diff completo antes de hacer commit.** Tú firmas el commit y respondes en la revisión, no el agente.
3. **Pide evidencia, no promesas:** «corre las pruebas y muéstrame la salida».
4. **Las reglas de negocio no se negocian.** Si el agente borra datos, asigna código l a un comodato o guarda salidas de IA sin aprobación humana, está mal aunque las pruebas pasen.

SECCIÓN 4

GitHub Flow

Hay **una sola rama permanente, main**, que siempre está estable. Todo el trabajo se hace en ramas cortas y entra a main por un Pull Request revisado y con CI en verde.



1. **Nadie hace push directo a main.** Toda rama sale de main actualizado: `git switch main && git pull`.
2. **Ramas con prefijo:** `feat/<change>` (el nombre exacto del change), `fix/...`, `docs/...`, `chore/...`
3. **Commits pequeños en Conventional Commits, en inglés:** `feat(catalog): add piece validation endpoint`, `fix(import): keep leading zeros in codes`, `chore(openspec): archive ficha-pieza-crud`.
4. **Abre el PR temprano como borrador (Draft):** CI corre desde el principio y el equipo ve el avance.
5. **Se integra con CI en verde y una aprobación** del Integrador de tu célula. Nadie aprueba su propio PR.
6. **Tras el merge se borra la rama** y se archiva el change en un PR pequeño aparte.

El Pull Request

Título en Conventional Commit. En la descripción, tres cosas: el **change**, los **IDs cubiertos** (RF, RN...) y la **evidencia** (`npm run lint`, `npm test`, `openspec validate --strict` y, si tocaste la base de datos, la prueba con Docker).

Job de CI	Qué verifica	Pruébalo antes en local
API	ruff, pytest, OpenAPI y diagrama ER al día	<code>npm run lint:api</code> , <code>npm run test:api</code> , <code>npm run openapi:check</code>
IA asistiva	ruff, pytest, OpenAPI al día	<code>npm run lint:ai</code> , <code>npm run test:ai</code>
Web	eslint, TypeScript, Vitest, build	<code>npm run lint:web</code> , <code>npm run test:web</code> , <code>npm run build:web</code>
OpenSpec	Specs y changes válidos (--strict)	<code>npm run validate:specs</code>

Mantén tu rama al día con main (antes de pedir revisión):

```
git fetch origin && git rebase origin/main # resuelve conflictos y: git rebase --continue
npm test
git push --force-with-lease               # nunca --force a secas
```

Trabajo en paralelo: tres reglas para no pisarnos

- **Migraciones:** después del rebase, tu migración de Alembic debe colgar de la última de main (`alembic heads` = una sola cabeza).
- **Contrato de la API:** `docs/fuentes/endpoints-api-v1.yaml` manda. Añadir o implementar operaciones es libre; renombrar o quitar requiere al Arquitecto.
- **Interfaces compartidas** (mapeo, reversión, exportación a Excel, aprobación de IA) tienen una célula dueña (`docs/ownership.md`). Si no está lista, usa una versión provisional.

Transición: hoy existe la rama develop

El arranque se trabajó en **develop**, que aún no está en main. La propuesta es integrar develop en main con un PR, proteger main (PR obligatorio, CI en verde, una aprobación) y desde ahí crear todas las ramas desde main.

Pendiente de confirmar por el Líder de Proyecto y el Arquitecto.

SECCIÓN 5

El flujo completo en 8 pasos

Ejemplo con `ficha-pieza-crud`. En OpenCode cambia `/opsx:` por `/opsx-`.

1 Toma el change — Asígnate la tarjeta en GitHub Projects y lee el change.

```
git switch main && git pull
openspec show ficha-pieza-crud      # dudas: /opsx:explore ficha-pieza-crud
```

2 Crea la rama

```
git switch -c feat/ficha-pieza-crud
```

3 Implementa — El agente marca cada tarea con su prueba. Revisa cada diff.

```
/opsx:apply ficha-pieza-crud
```

4 Commit y PR borrador

```
git add -A && git commit -m "feat(catalog): add piece validation endpoint"
git push -u origin HEAD
gh pr create --draft --base main
```

5 Verifica en local — Si tocaste la base de datos, también con Docker.

```
npm run lint && npm test && npm run openapi:check
openspec validate ficha-pieza-crud --strict
npm run dev && npm run migrate && npm run seed      # si hay migraciones
```

6 Pide revisión — Al día con main, descripción completa y al Integrador de tu célula.

```
git fetch origin && git rebase origin/main && git push --force-with-lease
gh pr ready
```

7 Integra — Con CI en verde y aprobación.

```
gh pr merge --delete-branch
```

8 Archiva — Solo después del merge, en un PR pequeño. Mueve la tarjeta a Hecho.

```
git switch main && git pull && git switch -c chore/archive-ficha-pieza-crud
openspec archive ficha-pieza-crud -y # o /opsx:archive
git commit -am "chore(openspec): archive ficha-pieza-crud" && git push -u origin HEAD
```

SECCIÓN 6

Antes de pedir revisión

Si puedes marcar todo esto, tu PR debería pasar la revisión a la primera. Los revisores usan la misma lista.

- ☐ Trabajé solo dentro de mi change, sin cambios «de paso».
- ☐ Cada tarea hecha está marcada - [x] y cada escenario WHEN/THEN tiene su prueba.
- ☐ `npm run lint`, `npm test` y `openspec validate <change> --strict` en verde.
- ☐ Si cambié la API: `npm run openapi` y `npm run openapi:client`. Si hay migración: probada con Docker y una sola cabeza.
- ☐ Respeté las reglas de negocio: código I inmutable, comodato sin I, nunca borrar (soft-delete + auditoría), IA con aprobación humana.
- ☐ Supuestos marcados [SUPUESTO] y registrados; decisiones técnicas nuevas como ADR *Propuesto*; manual de usuario actualizado si cambió una pantalla.
- ☐ Solo datos sintéticos: nada real del museo en código, fixtures ni capturas.
- ☐ Título Conventional Commit y descripción con change, IDs y evidencia.

Nunca	En su lugar
Programar sin change	Proponer el change primero y coordinar el alcance con el Líder de Proyecto.
Push directo a main o <code>git push --force</code>	Rama + PR; <code>git push --force-with-lease</code> .
Archivar antes del merge	Archivar después, en un PR aparte.
Aceptar el código del agente sin leerlo	Leer el diff completo y pedir la salida de las pruebas.
Dejar una rama viva semanas	PRs más pequeños y rebase frecuente sobre <code>origin/main</code> .

¿Dudas? Pregunta en el lugar correcto

Negocio o museo → Líder de Proyecto ([docs/preguntas-contraparte.md](#)). Arquitectura, API o migraciones → Arquitecto. Interfaz compartida → líder de la célula dueña. CI o PR → Integrador.

ANEXO

Chuleta de comandos

Momento	Comando
Ver el backlog	<code>openspec list · openspec show <change></code>
Empezar	<code>git switch main && git pull && git switch -c feat/<change></code>
Agente: dudas / implementar	<code>/opsx:explore · /opsx:apply (OpenCode: /opsx-explore · /opsx-apply)</code>
Sin comandos slash	<code>openspec instructions apply --change <change> · openspec status --change <change></code>
Entorno	<code>npm run dev · npm run migrate · npm run seed · npm run down</code>
Probar y lintear	<code>npm test · npm run lint</code>
Contrato de la API	<code>npm run openapi · npm run openapi:client · npm run openapi:check</code>
Validar specs	<code>openspec validate <change> --strict</code>
Al día con main	<code>git fetch origin && git rebase origin/main && git push --force-with-lease</code>
PR	<code>gh pr create --draft --base main · gh pr ready · gh pr checks</code>
Integrar	<code>gh pr merge --delete-branch</code>
Archivar (tras el merge)	<code>openspec archive <change> -y</code>

Para profundizar

Tema	Dónde
Reglas de negocio, dominio y convenciones	<code>CLAUDE.md</code>
Preparar el entorno y flujo diario	<code>docs/ONBOARDING.md</code>
Qué le toca a mi célula	<code>docs/ownership.md</code>
Requisitos (RF, RNF, RIA, RN)	<code>docs/requisitos/catalogo.md</code>
Decisiones técnicas	<code>docs/adr/</code> (ADR-000 comandos, ADR-002 idioma, ADR-010 trabajo en paralelo)
OpenSpec · OpenCode · GitHub Flow	<code>github.com/Fission-AI/OpenSpec · opencode.ai/docs · docs.github.com/get-started/using-github/github-flow</code>